PHASE 2 · WEEKS 4–6 · MULTI-SOURCE RETRIEVAL

Step-by-Step Project Guide

From three empty databases to a DSPy-compiled synthesizer that reasons across all of them at once.

Companion to: Phase 2 — Problem Statement

AI Engineering Masters Program

How to Use This Guide

This build has three independent data stores before it has any intelligence layer. Get each store working and independently queryable first — do not start on DSPy synthesis until all three engines pass their own smoke test.

1 Design Your Entity Schema

- Pick your domain from the Problem Statement and define the core entity (e.g. a startup, a player, a supplier).
- Decide what lives in SQL (structured numeric/categorical facts), what lives in FAISS (free text you'd search semantically), and what lives in Neo4j (relationships between entities).
- Checklist: a one-page schema note exists covering all three stores before you write any code.

2 Build the Synthetic Data Generator

- Write one script that populates all three stores from a manifest of ~20–30 fake entities — mirror `generate_sandbox.py`'s structure.
- SQLite: create tables, insert structured rows.
- FAISS: chunk and embed free-text content locally, persist the index and a metadata pickle/JSON.
- Neo4j: run via Docker; write nodes and typed relationship edges.
- Checklist: running the script once populates all three stores and prints row/vector/node-edge counts, like the reference project's console output.

3 Query Each Engine Independently

- Write `query_sql(entity)`, `query_faiss(entity)`, `query_graph(entity)` as separate, independently testable functions.
- FAISS query should combine exact metadata match with semantic nearest-neighbor fallback (hybrid retrieval — this is your BM25/RRF-equivalent moment; you may use real BM25 + dense fusion instead if you prefer).
- Each function should return a clearly labeled context string, and a tagged error string (not a crash) if its store is empty or unreachable.
- Checklist: each of the three functions runs standalone from a Python shell and returns sane output for a known test entity.

4 Parallelize Retrieval

- Wrap all three query functions in a `ThreadPoolExecutor` (or asyncio.gather) call that fires them simultaneously.
- Confirm total latency is close to the slowest single engine, not the sum of all three — that's your proof parallelism is real.
- Checklist: stop Neo4j and confirm the parallel call still returns SQL + FAISS context with a tagged error for the graph, instead of crashing.

5 Define the DSPy Signature & Module

- Define a DSPy `Signature` with your three context strings as inputs and your structured verdict fields as outputs.
- Wrap it in a `dspy.ChainOfThought` module so the model reasons before committing to a verdict.
- Configure a teacher model (stronger/more expensive) and a student model (cheaper) via your LLM provider.

- Checklist: an uncompiled call to the module returns syntactically valid structured output for at least one test entity.

6 Write the Judge Metric

- Write a metric function that parses the module's output and returns True/False based on: valid structure, required fields present, values in valid ranges, and at least one hard content rule (e.g. banned words, as in the reference project).
- Checklist: manually construct one deliberately bad output and confirm the judge correctly rejects it.

7 Compile with BootstrapFewShot

- Build a training set by running parallel retrieval across your full entity manifest and wrapping each result as a `dspy.Example`.
- Run the optimizer with your judge metric; save the compiled state to disk so future runs load it instead of recompiling.
- Checklist: a saved optimized-state file exists, and you can show at least one example of the student's reasoning trace after compilation.

8 Analyze & Compare

- Run the compiled module against a held-out entity not in the training manifest.
- Capture one side-by-side example of uncompiled vs. compiled output quality for your README.
- Checklist: compiled output is measurably more consistent/structured than the uncompiled baseline — show this, don't just claim it.

9 Build the Transparency UI

- Show all three raw contexts (SQL / FAISS / Graph) side by side, plus the final synthesized verdict — the point is to make the multi-engine retrieval visible, not hidden behind a chatbot.
- Checklist: a reviewer can pick any entity and see exactly what each of the three engines contributed to the verdict.

10 Write the README & Deploy

- Document the architecture diagram (mirror the reference project's ASCII diagram style), tech stack table, and full setup for all three databases including Docker commands for Neo4j.
- Deploy per the Vibe Coding Helper Doc; note clearly in the README if Neo4j must run locally / via a hosted graph DB for the deployed version to fully work.
- Checklist: setup instructions were followed by someone other than you, end to end, without needing your help.